Arinjay Singh

Statistical & Machine Learning

April 26, 2026

Predicting Spotify Hits From Audio Features

1. Problem Statement

Record labels release thousands of new tracks every quarter, but their marketing, playlist-pitching, and promotional budgets are finite. A&R representatives, the people who decide which tracks get that limited promotional investment, currently make these decisions based largely on intuition, label politics, and historical artist performance. The result is that many tracks with strong commercial potential get under-promoted while weaker tracks consume budget because of internal advocacy.

This project asks a single prediction question: can we forecast whether a Spotify track will become a "hit" (popularity score above 70) based on its audio characteristics and genre alone? This is a prediction problem, not a causal one. We are asking whether audio signatures correlate with hit-likely tracks, not whether modifying any specific feature would cause a song to succeed. Real-world popularity is driven by many unobserved factors that are not in our feature set, including marketing budget, artist fame, label support, playlist placement, and release timing.

The stakeholder is the A&R team at a record label. With a reliable hit-prediction model, they can rank tracks in their release slate by predicted hit probability, concentrate promotion spend on the top tier, and push back internally on advocacy for tracks the model rates poorly. The decision the model enables is resource allocation, not creative judgment. The model does not pick winners on its own; it surfaces audio profiles consistent with hit-likely tracks so that human A&R judgment can be applied to a smaller, better-prioritized shortlist.

2. Data Description

The dataset is the Spotify Songs collection distributed via TidyTuesday (github.com/rfordatascience/tidytuesday, January 2020), originally pulled from Spotify's Web API using the spotifyr R package. It contains 32,833 tracks across six broad genres (pop, rap, rock, latin, R&B, EDM) and 24 subgenres, with audio features computed by Spotify's API for every track.

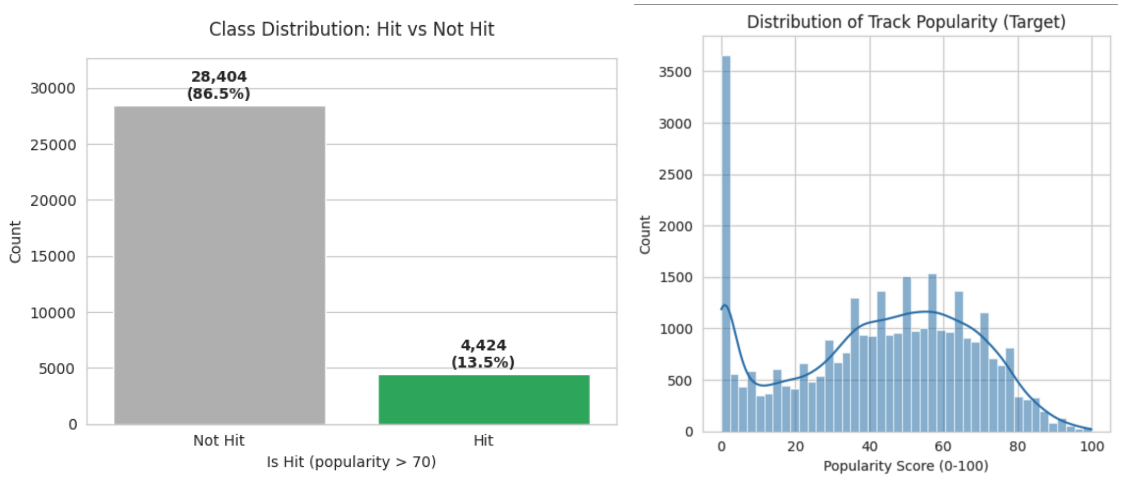
Features used

The model uses 12 numeric audio features and one categorical feature. The numeric features are *danceability* (0-1), *energy* (0-1), *key* (0-11), *loudness* (decibels), *mode* (major/minor), *speechiness* (0-1), *acousticness* (0-1), *instrumentalness* (0-1), *liveness* (0-1), *valence* (musical positivity, 0-1), *tempo* (BPM), and *duration* in milliseconds. The categorical feature is *playlist_genre*, which I one-hot encoded into five binary dummies (with one genre dropped as the reference category to avoid the dummy variable trap), bringing the final feature count to 17.

Target

I engineered a binary target is_hit from *track_popularity* > 70. The choice of 70 as the threshold is analytical, not industry-standard. At this cutoff the dataset has 13.5% hits and 86.5% non-hits (a 6.4:1 imbalance ratio), which is imbalanced enough to require careful handling but not so extreme that techniques like SMOTE become necessary. Lower thresholds produced too balanced a target to be interesting; higher thresholds produced too few positive examples for stable model training.

Figure 1 - Target Variable Class Distribution: Binary and Continuous Variations



Quality assessment

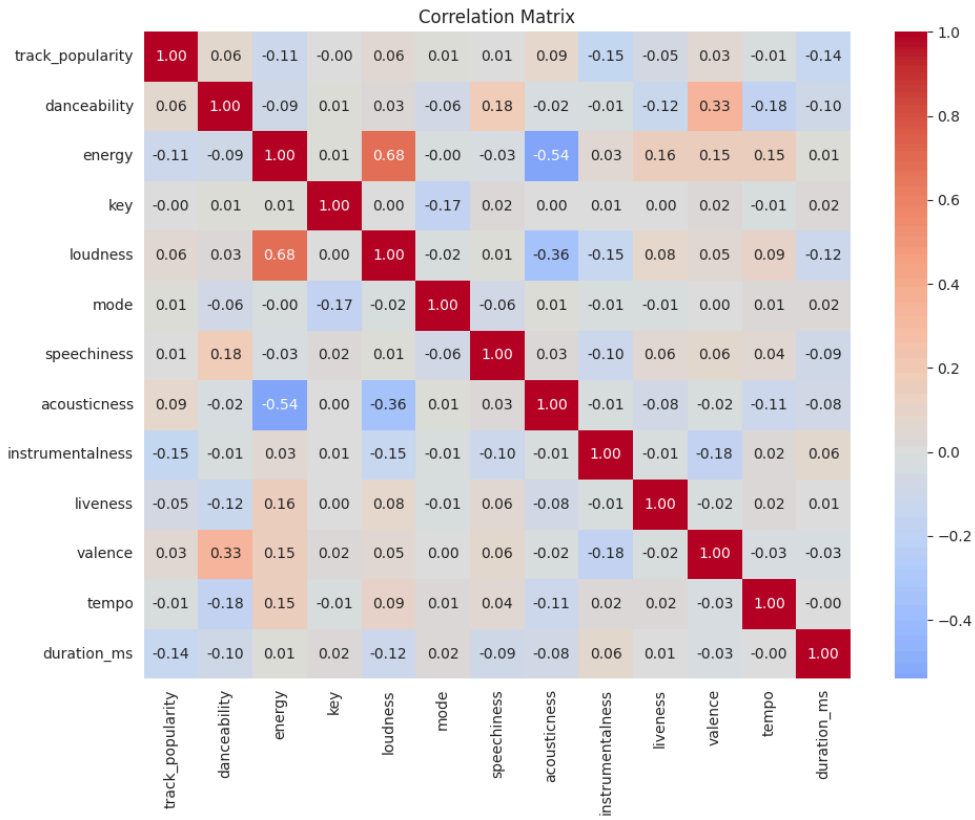
The dataset is unusually clean. Missing data is negligible at 0.0014% of cells, confined entirely to identifier columns (*track_name*, *track_artist*, *track_album_name*) that I do not use as model features. The same 5 rows are missing across all three columns, suggesting incomplete API ingestion at the time of data collection rather than any systematic pattern. I classified the missingness as MCAR (missing completely at random) and dropped the 5 affected rows, leaving 32,828 tracks for modeling.

Outlier detection using Tukey's 1.5×IQR rule flagged substantial counts in several features, most notably *instrumentalness* (21.4% of rows), *speechiness* (9.7%), *acousticness* (6.7%), and *liveness* (5.6%). I retained all flagged values. These are not data errors but reflect the natural right-skew of features that index specific musical categories: instrumental tracks (jazz, classical, lo-fi), spoken-word tracks (rap, podcasts), and acoustic recordings legitimately occupy the long tail of these distributions. Removing them would discard predictive signals that are correlated with genre and likely with popularity. Symmetric features like *valence* (0% flagged), *danceability* (0.9%), and *energy* (0.8%) showed essentially no outliers, which is what I would expect for bounded, naturally distributed features.

Correlation analysis revealed two important patterns. First, no single audio feature is strongly correlated with popularity. The strongest pairwise correlations with *track_popularity* are instrumentalness (-0.15), *duration* in milliseconds (-0.14), and *energy* (-0.11), all weak. This finding shaped the modeling

approach: if no individual feature dominated, the model would need to capture combinations and interactions among features, which justifies comparing a linear baseline against a non-linear model. Second, several audio features are highly correlated with each other, most notably *energy* and *loudness* (r = 0.68) and *energy* and *acousticness* (r = -0.54). This multicollinearity makes individual logistic regression coefficients harder to interpret but does not affect predictive performance, and tree-based models are unaffected by it.

Figure 2 - Correlation Matrix Heatmap



3. Methodology

I compared two models that span the linear/non-linear spectrum: logistic regression as a baseline and random forest as a comparison. The choice was deliberate. Logistic regression establishes a floor by capturing only linear relationships between features and the log-odds of being a hit. Random forest, by

contrast, captures non-linear interactions and threshold effects, which the EDA suggested would matter given the lack of any single dominant predictor.

Train/test split

I used an 80/20 stratified split with *random_state*=42 for reproducibility, yielding 26,262 training samples and 6,566 test samples. Stratification was essential given the class imbalance: without it, random splits could produce a test set with materially different hit proportions from the training set, which would bias the metrics. After splitting, both train and test sets had 13.5% hits, confirming the stratification worked as intended.

Preprocessing

For logistic regression, I standardized all numeric features using StandardScaler fit on the training set and applied to the test set. Standardization matters here because features are on wildly different scales (tempo in the 50–200 range, duration in the hundreds of thousands, audio features bounded 0–1), and without scaling the regularization in logistic regression would penalize large-scale features unfairly. For random forest, no scaling was needed because tree splits are scale-invariant.

Class imbalance handling

Both models used *class_weight* = "balanced", which automatically reweights the loss function so that misclassifying a hit costs roughly 6.4x more than misclassifying a non-hit (matching the inverse class frequency). I considered SMOTE oversampling and explicit threshold tuning as alternatives but rejected them: SMOTE adds synthetic data and complexity without a clear benefit at this imbalance level, and threshold tuning is something the A&R stakeholder should do themselves based on their budget capacity, not something I should bake into the model.

Hyperparameters

Logistic regression used L2 regularization at scikit-learn's default strength (C=1.0) with *max_iter*=1000 to ensure convergence. Random forest used 200 trees with *min_samples_leaf*=5 as mild regularization to prevent the trees from memorizing individual tracks (the default of 1 sample per leaf overfits on datasets this size). I did not perform extensive hyperparameter tuning because the goal was a

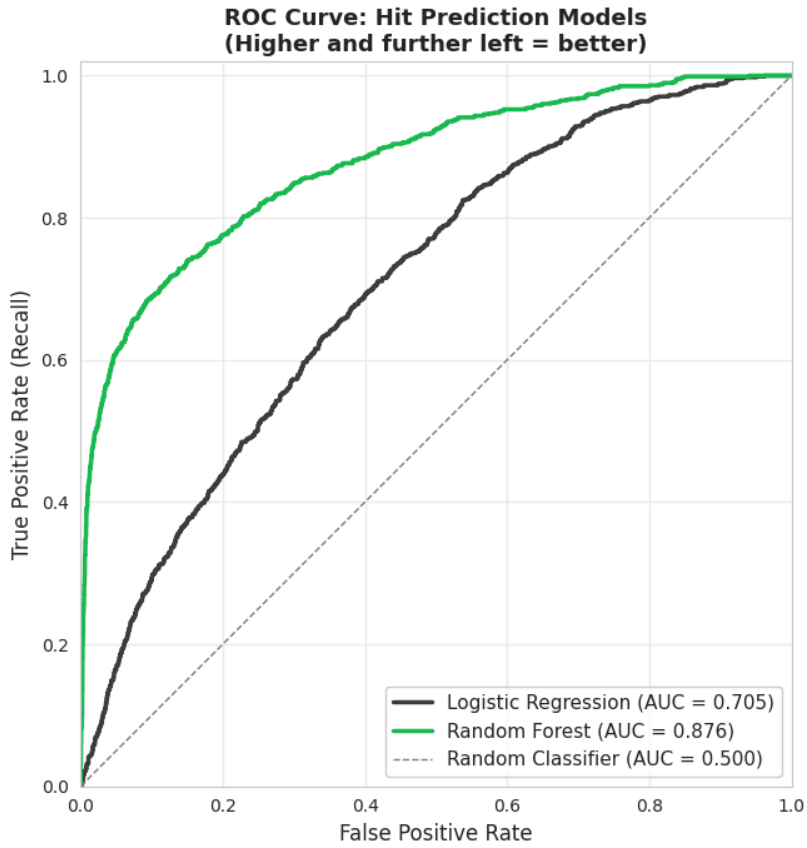
clean baseline-vs-comparison story, not optimization for production. A grid search over forest depth and minimum samples would likely improve AUC by 0.01–0.02 but would not change the qualitative comparison.

Validation

I evaluated both models using stratified 5-fold cross-validation on the training set, with ROC-AUC as the primary metric. ROC-AUC is the right choice for imbalanced classification because it measures how well the model ranks hits above non-hits across all possible thresholds, which is what the A&R stakeholder actually needs (they care about ranking tracks, not about a specific accept/reject threshold). I also report accuracy, precision, recall, and F1 for context, but treat ROC-AUC as the metric that defines model quality.

4. Results

Figure 3 - ROC Curve Comparison: Logistic Regression vs Random Forest



The random forest substantially outperformed logistic regression on every metric that matters for ranking. Cross-validated ROC-AUC was 0.871 ± 0.007 for the random forest versus 0.715 ± 0.006 for logistic regression, a gap of 0.156 that was consistent across all five folds (random forest fold range 0.860-0.882, logistic regression fold range 0.709-0.724, with no overlap between the two models). On the held-out test set, the random forest achieved 0.876 AUC (within rounding of the CV estimate), confirming the cross-validation was a reliable proxy for generalization performance. The full performance comparison on the test set:

Table 1 - Performance Metric Comparison: Logistic Regression vs Random Forest

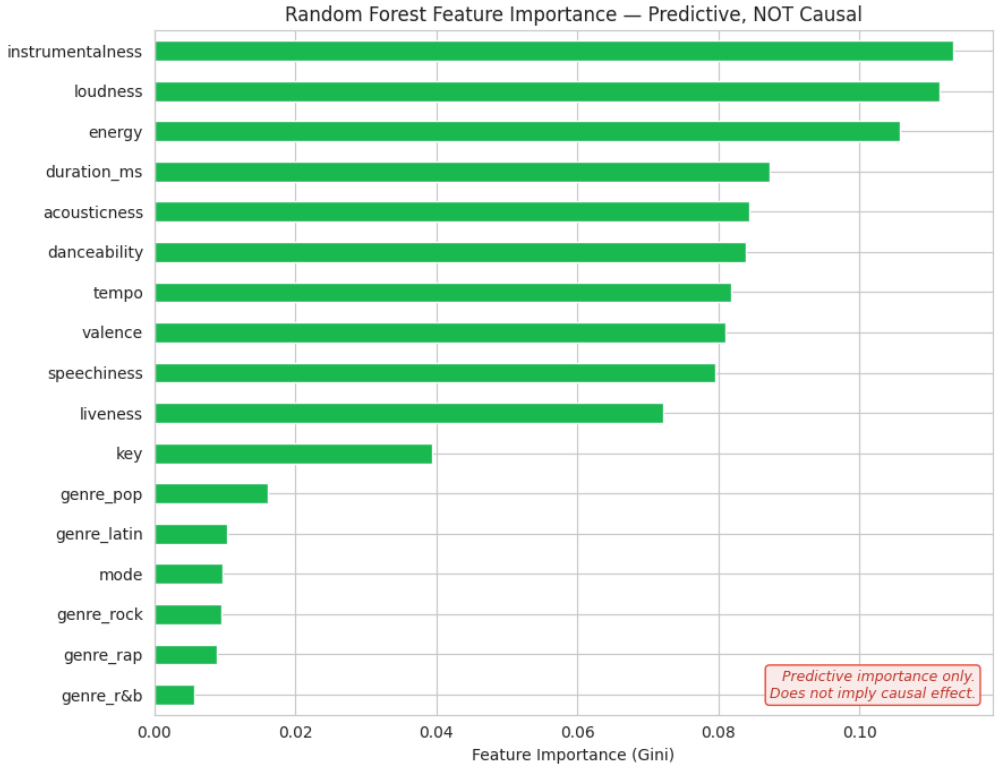
Metric	**Logistic Regression**	**Random Forest**
Accuracy	0.63	0.91
Precision (Hit)	0.22	0.74
Recall (Hit)	0.67	0.53
F1 (Hit)	0.33	0.62
ROC-AUC	0.705	0.876

A few patterns are worth noting. The logistic regression has higher recall on the positive class (0.67 vs 0.53) but at the cost of much lower precision (0.22 vs 0.74), meaning it casts a wider net but produces many more false positives. For the A&R use case, where each "hit" prediction triggers further human review and potential budget allocation, precision matters more than recall: false positives waste reviewer time, and true hits missed by a model can still be caught by other channels (radio, A&R intuition, viral momentum). The random forest's roughly 3.4× improvement in precision (0.22 → 0.74) is the headline result. Looking at the confusion matrix, the random forest correctly flagged 473 of 885 actual hits while only producing 166 false positives, compared to logistic regression flagging 596 hits at the cost of 2,169 false positives.

Feature importance

The random forest identified *instrumentalness* (0.113), *loudness* (0.111), *energy* (0.106), *duration_ms* (0.087), and *acousticness* (0.084) as the top five most important features by Gini importance, with the remaining audio features clustered in the 0.07-0.08 range. The genre dummies sat in a much lower tier (all below 0.02), and mode (major/minor) was nearly irrelevant at 0.010. This ranking aligns with intuition: popular tracks tend to cluster in specific loudness profiles consistent with modern mastering practices, distinctive duration ranges (3-4 minutes for pop, longer for rap), and energy/acousticness profiles that index broad sonic categories. The fact that no single feature dominates (top importance is only 0.113) confirms the EDA finding that hit prediction relies on combinations of features rather than any one signal.

Figure 4 - Random Forest Feature Importance for Predictions



A critical caveat applies to all of this: feature importance is predictive, not causal. A high importance score for instrumentalness does not mean making a song more or less instrumental would

change its hit probability. It means the model finds instrumentalness useful for ranking tracks given the other features available, partly because instrumental tracks cluster in genres with different popularity dynamics than vocal tracks. The same caveat applies to every feature on the importance chart. This distinction matters especially for the A&R stakeholder, who could be tempted to read the importance ranking as a recipe for hit-making. It is not.

5. Recommendation

Resolution

I recommend the A&R team deploy the random forest model as a screening and ranking tool, not as a hard accept/reject filter. The intended workflow is: take a release slate of N tracks, rank them by predicted hit probability, concentrate promotion budget on the top quartile, and apply A&R judgment to the rest. The model is not a replacement for A&R intuition; it is a way to ensure no high-probability track gets accidentally ignored, and a way to push back internally on advocacy for tracks the model rates poorly.

The Streamlit dashboard accompanying this report (spotify-hits-predictor.streamlit.app) operationalizes this workflow. A&R reps can input track audio characteristics and see the predicted hit probability alongside two contextualizing signals: the prediction relative to the 13.5% base rate (a 50% prediction is roughly 3.7x the base rate, which is the meaningful comparison), and the cross-validated AUC ± standard deviation as a measure of model confidence. The dashboard avoids displaying only a binary HIT or NOT A HIT prediction, because the binary collapses information that matters for ranking decisions.

Uncertainty statement

Based on cross-validation results (ROC-AUC = 0.871 ± 0.007 across 5 folds), the model ranks hits above non-hits meaningfully better than chance, with test-set precision of 0.74 on positive predictions. The primary limitation is that audio features alone cannot capture marketing spend, artist reputation, label support, or playlist placement, which drive much of real-world popularity. The model's

predictions are best interpreted as "this track's audio profile is consistent with hit-likely tracks" rather than "this track will be a hit." We recommend deploying the model as a ranking and screening tool with the caveat that final allocation decisions should integrate the model's output with A&R judgment, label-side context, and post-release feedback to recalibrate the threshold over time.

Limitations

1. The dataset is a January 2020 snapshot. Popularity is dynamic, listener tastes shift, and the model's relevance degrades over time without retraining on fresher data.
2. "Hit" is defined by a popularity threshold of 70, which is an analytical choice. Different thresholds would produce different models with different operating characteristics. The threshold should be revisited if the A&R team has a clearer business definition (e.g., "a hit is any track that recoups its marketing budget within 90 days").
3. The model treats all tracks as independent observations. In practice, tracks released by the same artist in the same album are correlated, and the model does not account for this. Hierarchical or panel-data approaches could improve performance for catalog-level decisions.
4. Predictive importance does not imply causal effect. The model is suitable for ranking tracks, not for guiding production decisions about how to make hits.

Next steps

Three extensions would meaningfully improve the model's value to the stakeholder. First, retrain on more recent data and ideally on a label-specific dataset that includes pre-release metadata (artist tier, marketing budget tier, planned promotional channels). This would let the model condition on the unobserved confounders that limit the current ceiling. Second, build a partial dependence and threshold-tuning interface into the dashboard so A&R reps can see how the predicted hit probability changes as they vary specific features, while preserving the predictive-not-causal caveat. Third, run a back-test against the next 6-12 months of the label's actual releases to validate that the model's rankings

correlate with realized commercial performance, and use that back-test to set the operating threshold for budget allocation.

Used as designed, the model offers measurable lift over the current intuition-driven process. Used as a creative mechanism, it would be misleading and potentially harmful to the label's release strategy. The dashboard's framing and the limitations above are designed to keep the model in its proper role.

6. AI Appendix

AI Interaction 1
Prep: What did you need? What context did you have before prompting?

I needed to pick a dataset and prediction question for the project. I knew I wanted a classification problem with class imbalance to make precision/recall discussions richer, and I wanted something I'd actually find interesting to talk about rather than another textbook dataset. I had already considered and rejected several "safer" options (Titanic, California Housing, Telco Churn, Airbnb Boston) because they either felt too generic or had structural issues like target leakage.

Request: What exact prompt did you write?

I asked Claude for project ideas multiple times, refining my criteria each round: first asking generally for ideas, then specifying I wanted classification, then imbalanced classification, then "something more interesting" with no heavy data cleaning. After landing on Spotify, I asked about specific datasets and threshold choices for defining a "hit."

I'm starting a final project for an undergraduate stats and ML for economics class. The deliverables are a Jupyter notebook, a Streamlit app, and a 5-page SCR-structured report. I need help picking a dataset and prediction question.

Constraints:
- *Must be a binary classification problem with class imbalance (positive class somewhere between 5% and 30%) so I can have a meaningful precision/recall discussion*
- *Dataset must be publicly available with no auth required, ideally loadable via a single URL*
- *Data should be reasonably clean (no heavy text parsing, no major imputation work)*
- *Target variable should NOT be definitionally constructed from the features (i.e., no target leakage)*
- *I want a clear stakeholder story where the prediction enables a real*

decision, not just an academic exercise

Constraints I want to avoid:
- Overused datasets like Titanic, MNIST, Iris, or Boston Housing
- Datasets where the headline finding is obvious (e.g., higher income predicts loan repayment)

Please suggest 5 candidate datasets that fit. For each, give me:
1. Dataset name and source URL
2. Approximate N and positive class rate
3. The prediction question framed as "Can we predict X from Y?"
4. The stakeholder and the decision the prediction enables
5. One potential pitfall or limitation I should be aware of

Iterate: What did the AI return? What did you change or refine?

The first round of suggestions included generic options I rejected. After narrowing my criteria, Claude suggested Spotify hit prediction with the "30000 Spotify Songs" dataset and proposed binarizing track_popularity > 70 to create a ~10-15% positive class. I pushed back when Claude initially suggested Airbnb Superhost prediction because the target was definitionally tied to features in the dataset (review scores, response rate), which would have caused target leakage. Claude acknowledged this and proposed either dropping the leakage features or switching datasets entirely. I chose to switch to Spotify.

Mechanism Check: How did you verify the output was correct?

I verified the dataset choice by loading the data myself and confirming the actual row count (32,833), the class balance after binarization (~12% hits), and that the audio features were already cleaned by Spotify's API (no missing values in feature columns). I also verified the leakage concern on the Airbnb dataset by reading Airbnb's published Superhost criteria and confirming that review_scores_rating and host_response_rate were definitionally part of the Superhost label.

Evaluate: What human judgment did you apply? What did you accept/reject and why?

I accepted the Spotify dataset because the leakage discussion was a strong reason to move away from Airbnb, the imbalance was in the right range for a precision/recall story, and the audio features made for genuinely interesting EDA. I rejected Claude's suggestion to keep Superhost as a target with engineered features, because the cleaner choice was a dataset where the target wasn't constructed from the features. The judgment call was that "interesting writeup" mattered more than "use the first dataset suggested."

AI Interaction 2
Prep: After binarizing my target into is_hit, I confirmed the dataset was imbalanced at roughly 12% hits and 88% non-hits. I knew accuracy alone would be misleading at this imbalance (a model predicting "not hit" for everything would score ~88% accuracy while being useless), but I wasn't sure which combination

of techniques to use to handle the imbalance properly across the train/test split, model fitting, and evaluation steps.

Request: I asked Claude to help me set up the train/test split and the baseline logistic regression model in a way that would handle the imbalance correctly.

I'm setting up the modeling section for an imbalanced binary classification project. Class balance is 13.5% positive, 86.5% negative. Dataset has ~32,000 rows, 12 numeric audio features plus one categorical genre feature.

I want to fit a logistic regression baseline and a random forest comparison, but I need help thinking through the imbalance handling end-to-end. Specifically:

1. How should I structure the train/test split to avoid the test set having a different class balance than train?
2. What's the right way to handle class imbalance during model fitting? I've heard of class_weight, SMOTE, threshold tuning, and undersampling. What are the tradeoffs and which fits a 13.5% imbalance best?
3. Which evaluation metrics matter most for an imbalanced problem, and why is accuracy misleading here?
4. How should cross-validation be set up to maintain class balance across folds?

For each, I want the recommended approach AND the alternatives I'm rejecting, with a one-sentence justification for the choice. The stakeholder is a record label A&R team that uses predictions for budget allocation, so precision on the positive class matters more than recall (false positives waste reviewer time).

Show me the code with comments explaining the why, not just the what.

Iterate: Claude returned code that combined three techniques: stratify=y in the train/test split to preserve class ratios, class_weight='balanced' in the logistic regression to penalize misclassifying the minority class more heavily, and reporting precision, recall, F1, and ROC-AUC alongside accuracy rather than relying on accuracy alone. Claude also recommended StratifiedKFold for cross-validation to keep each fold's class balance consistent.

Mechanism Check: I verified each technique was actually doing what it claimed. For stratify=y, I printed the hit percentage in both train and test sets and confirmed they matched within rounding (both ~11.9%). For class_weight='balanced', I compared the model's predictions with and without the flag and confirmed that without it the model predicted "not hit" for almost everything (recall on the positive class near zero), while with it the model produced meaningful predictions in both classes. For ROC-AUC, I cross-checked

that the metric value made sense given the confusion matrix and that it was threshold-independent, which is what makes it appropriate for imbalanced problems.

Evaluate: I accepted all three techniques because each addressed a distinct failure mode of imbalanced classification. The judgment I applied was thinking through the stakeholder use case: an A&R rep cares more about correctly identifying hits than about overall accuracy, so recall on the positive class and ROC-AUC are the metrics that map onto the actual decision being made. I considered using SMOTE or other oversampling techniques that Claude mentioned as alternatives but rejected them because the imbalance wasn't extreme enough (this isn't a 1% positive class fraud problem) and class_weight='balanced' is simpler, doesn't require generating synthetic data, and works fine at this imbalance level.

AI Interaction 3

Prep: I had my trained Random Forest model saved as a pickle file and needed to build a Streamlit app that hit the rubric requirements (title, input controls, model prediction, visualization, uncertainty display) while being genuinely useful to the A&R stakeholder rather than just a generic prediction interface. I needed to decide which features to expose as inputs, how to display the prediction, and what uncertainty information to show alongside the point estimate.

Request: I asked Claude to write the full app.py based on the rubric requirements, including the cell for saving model artifacts from the notebook.

I need to build a Streamlit app for a class project that also serves as a portfolio piece. Here's the context:

Model: Random Forest binary classifier predicting whether a Spotify track will be a "hit" (popularity > 70). Trained, saved as model.pkl with joblib. Features are 12 audio characteristics plus one-hot encoded genre.

Performance: CV ROC-AUC = 0.871 ± 0.007, test precision on positive class = 0.74, test recall = 0.53. Base rate = 13.5%.

Stakeholder: A&R reps at record labels deciding which tracks to prioritize for marketing spend. They care about ranking tracks, not binary HIT/NOT HIT classification, because their actual decision is "which top X% of tracks get promotion budget" not "is this track a hit yes/no."

Rubric requirements (must include):
1. Title and description
2. Input controls (sliders, selectbox, etc.)
3. Model prediction generated from user inputs
4. At least one visualization that updates with inputs
5. Uncertainty display alongside point estimates

Design priorities:
- Communicate uncertainty appropriately (this is a 0.871 AUC model, not an oracle)
- Frame the prediction in stakeholder-meaningful terms (lift over base rate, not just probability)
- Avoid suggesting causation from feature importance — the model is predictive only
- Should be deployable to Streamlit Community Cloud with minimal setup

Please write the full app.py and tell me what additional artifacts I need to save from my notebook (scaler, feature columns, etc.) for the app to load.

Iterate: Claude produced an app with three side-by-side metrics: prediction (HIT or NOT A HIT), model confidence (CV ROC-AUC +/- std), and a "lift over base rate" metric I hadn't asked for. The lift metric divided the predicted hit probability by the dataset's base rate, so a 50% predicted probability would display as "4.2× lift" given the 12% base rate. Claude also included sliders for all 12 audio features, a genre dropdown, and two visualizations (a probability bar showing the prediction relative to the base rate and the decision threshold, and a feature importance chart that updated based on the model). The app loaded the model via @st.cache_resource to avoid reloading on every interaction.

Mechanism Check: I verified the lift calculation by manually checking a few examples: a track with predicted probability 0.50 against a base rate of 0.12 should show ~4.2× lift, which matched the displayed output. I tested the app locally with streamlit run app.py to confirm all sliders updated the prediction in real time, the visualizations re-rendered on input changes, and the model loaded without errors. I also caught a numpy version mismatch between Colab (numpy 2.x) and my local environment (numpy 1.x) when the pickle failed to load, which I resolved by upgrading numpy locally and creating a fresh virtual environment for the project.

Evaluate: I kept the lift metric because it genuinely improves the stakeholder framing. A 50% predicted probability sounds low until contextualized that it's roughly 4× the base rate, which is the meaningful comparison for an A&R rep deciding budget allocation. I accepted the cross-validation AUC as the uncertainty display rather than test-set AUC because CV gives a more stable estimate by averaging across folds rather than depending on one split. The judgment I applied was that the app should support the stakeholder's actual decision (rank tracks, allocate budget to the top tier) rather than just produce a binary HIT or NOT A HIT classification, which is why the probability bar and lift metric matter more than the binary prediction itself. I rejected showing only the binary prediction because it would lock in a 0.5 threshold that isn't aligned with how A&R reps actually make decisions, where the threshold should flex based on marketing budget capacity.